

Kafka Advanced Interview Workbook

100 Scenario-Based Questions • Cross Questions • Java/Spring Kafka Coding • Production Incidents

Target: 8–12 year Java/Spring Boot Senior Developer, Tech Lead and Architect interviews

Purpose: Master Kafka from internals to architecture, coding, troubleshooting and leadership-level trade-offs.

How to use: Answer aloud first. Then compare with the model answer. For every scenario, practice the cross-question until you can defend the trade-off, failure mode and operational impact.

How to Approach Kafka Interviews at Tech Lead Level

Interviewers are usually testing more than Kafka terminology. They want to know whether you can design a reliable distributed system and reason about failure.

- **Requirement first:** throughput, latency, ordering, durability, replay, fan-out and business SLA.
- **Partition strategy:** identify the business ordering boundary and choose the key.
- **Delivery semantics:** explain what happens on crash, retry and replay.
- **Failure design:** timeout, retry, DLT, backpressure, idempotency and recovery.
- **Operations:** lag, rebalances, broker health, disk, network, replication and capacity.
- **Trade-offs:** never give a configuration as a universal answer.

Golden response structure: Requirement → Assumptions → Design → Why → Failure modes → Trade-offs → Monitoring → Recovery.

Part I — Core Kafka & Internals Scenarios (1–20)

1. Explain Kafka to an interviewer without calling it “just a queue.”

Model answer: Kafka is a distributed append-only log. Producers append records to partitioned topics; consumers track offsets and can replay records. Partitions provide horizontal scalability and the boundary of ordering. Consumer groups distribute work, while replication provides fault tolerance.

Cross questions: Why is replay important? How is Kafka different from a traditional queue?

2. A topic has 12 partitions and 30 consumers in one group. What happens?

Model answer: At most 12 consumers can actively own partitions at a time. The remaining consumers are idle. Adding consumers beyond partition count does not create additional partition parallelism.

Cross questions: What if we need 30-way parallelism? What are the risks of simply increasing partitions?

3. Why are partitions the fundamental scaling unit?

Model answer: A partition is an ordered log that can be independently stored, fetched and processed. Different partitions can be handled concurrently by brokers and consumers.

Cross questions: What limits scaling? Can one partition be processed by two consumers in the same group simultaneously?

4. A business requirement says: “All events for the same order must be processed in order.” Design it.

Model answer: Use `orderId` as the record key so related records normally map to the same partition. The ordering boundary becomes the order, not the entire topic.

Cross questions: What if one order becomes extremely hot? What if global ordering is required?

5. Can Kafka guarantee global ordering?

Model answer: No. Kafka guarantees ordering within a partition. Global ordering would require a single partition or another coordination mechanism, which sacrifices parallelism.

Cross questions: Why is one partition a scalability bottleneck?

6. What is an offset?

Model answer: An offset is the position of a record in a partition. A consumer group uses offsets to track progress and enable replay or resume after failure.

Cross questions: Committed offset vs current consumer position?

7. A consumer crashes after processing but before committing the offset. What happens?

Model answer: The record can be processed again after reassignment. This is why at-least-once processing normally requires idempotent business handling.

Cross questions: How would you prevent duplicate DB transactions?

8. Why is consumer lag a symptom rather than a root cause?

Model answer: Lag tells us the consumer is behind the log. The cause may be higher producer rate, slow processing, hot partitions, downstream latency, rebalances, or Kafka infrastructure issues.

Cross questions: What metrics would you inspect first?

9. What is a consumer group?

Model answer: A set of consumers sharing a group ID. Kafka assigns partitions among group members so each partition is actively owned by one member at a time.

Cross questions: Can two different groups consume the same topic?

10. Why might two consumer groups have very different lag on the same topic?

Model answer: They may have different processing times, downstream dependencies, partition assignment, concurrency, code versions or retry behavior.

Cross questions: How would you compare them operationally?

11. What is ISR?

Model answer: In-sync replicas are replicas sufficiently caught up with the partition leader under Kafka's replication rules. ISR is important for durability and safe failover.

Cross questions: How does min.insync.replicas relate to ISR?

12. Does replication factor 3 guarantee zero data loss?

Model answer: No. RF=3 improves durability but does not itself guarantee zero loss. Producer acknowledgement, min.insync.replicas, failure mode and cluster configuration matter.

Cross questions: What happens when ISR falls below min.insync.replicas with acks=all?

13. What does acks=all mean?

Model answer: The producer waits for the required in-sync replica acknowledgement according to the broker configuration. It is stronger durability than acks=1 or 0.

Cross questions: Is acks=all the same as exactly-once?

14. What does producer idempotence solve?

Model answer: It prevents duplicate records caused by producer retries within Kafka's idempotent producer semantics.

Cross questions: Does it prevent duplicates caused by the application sending the same business event twice?

15. Why can retries make an outage worse?

Model answer: Retries add traffic and processing work while the dependency is already unhealthy. Unbounded retries can create retry amplification and increase lag.

Cross questions: How do you design bounded retry?

16. Retention vs compaction — when would you use each?

Model answer: Retention preserves a replayable history based on time/size. Compaction preserves the latest state for keys, subject to compaction semantics.

Cross questions: Can a topic use both? Give an example.

17. Why is increasing partitions not always a safe change?

Model answer: It changes the available parallelism and can affect key-to-partition mapping for future records. Ordering assumptions, consumers, storage and operational overhead must be reviewed.

Cross questions: Would existing records be redistributed?

18. What is a hot partition?

Model answer: A partition carrying disproportionately high traffic or processing work, often caused by skewed keys.

Cross questions: Why doesn't adding consumers necessarily fix it?

19. What happens during a consumer rebalance?

Model answer: Partition ownership is redistributed among group members. Processing can pause or become less efficient, and unstable groups can repeatedly create lag.

Cross questions: What causes frequent rebalances?

20. Why is Kafka not a database?

Model answer: Kafka is a durable distributed log/event platform, not a general-purpose query/update database. It can retain and replay events but does not replace relational consistency, indexing and transactional querying.

Cross questions: Where would you use an outbox pattern?

Part II — Producer, Consumer and Delivery Semantics (21–40)

21. Your producer sends critical financial events. What configuration and design do you consider?

Model answer: Use appropriate serializers, strong acknowledgements such as `acks=all`, idempotence, bounded retries/timeouts, suitable batching/compression and a carefully chosen business key. Pair this with schema governance and monitoring.

Cross questions: Why is idempotence important? What does it not solve?

22. A producer receives a timeout. Did Kafka definitely lose the message?

Model answer: Not necessarily. A timeout means the client did not receive a successful response within the configured period. The broker may have processed the request. Retrying can therefore create duplicates unless producer idempotence/transaction semantics handle the case.

Cross questions: Why are ambiguous outcomes important in distributed systems?

23. What is the difference between request timeout and delivery timeout?

Model answer: Request-level timeout limits an individual request interaction; delivery timeout represents the overall time the producer allows for delivery attempts before failing the send.

Cross questions: How do retries interact with these settings?

24. Why use batching?

Model answer: Batching reduces per-record protocol overhead and can improve throughput and compression efficiency. It can add small latency because the producer waits to accumulate records.

Cross questions: Which settings influence batching?

25. Why use compression?

Model answer: It reduces network traffic and storage consumption, often improving throughput. The trade-off is CPU cost and payload characteristics.

Cross questions: Would compression always reduce end-to-end latency?

26. A consumer must process 100,000 records quickly. What would you tune first?

Model answer: Measure first. Check partition count, consumer parallelism, processing latency, downstream capacity, `max.poll.records`, batching opportunities and CPU/GC. Do not tune blindly.

Cross questions: What if DB is the bottleneck?

27. What is the risk of setting `max.poll.records` extremely high?

Model answer: Large batches can increase memory use and processing time. If processing exceeds the allowed poll interval, the consumer can be considered unresponsive and trigger a rebalance.

Cross questions: How would you choose a value?

28. When would you increase `max.poll.interval.ms`?

Model answer: When measured, legitimate record processing takes longer than the current interval. It should accommodate expected work, not conceal an inefficient or blocked consumer.

Cross questions: What other settings affect group stability?

29. Explain at-most-once.

Model answer: Commit/advance the consumer position before processing. A crash after the commit can cause the record not to be processed, so loss is possible but duplicate processing is reduced.

Cross questions: Where is this acceptable?

30. Explain at-least-once.

Model answer: Process first and commit after successful processing. A crash between processing and commit can cause the record to be processed again, so idempotency is essential.

Cross questions: Why is this common in business systems?

31. Explain exactly-once in Kafka.

Model answer: Kafka transactions can atomically publish records and commit consumer offsets in a compatible consume-transform-produce workflow. Exactly-once is a scoped system property, not a guarantee for arbitrary external side effects.

Cross questions: Why doesn't it make a PostgreSQL call exactly-once?

32. Design idempotent consumer processing for payment events.

Model answer: Use a stable event/payment transaction ID, persist a processed marker or unique business key transactionally with the business update, and acknowledge/commit only after successful DB work.

Cross questions: Why is a DB unique constraint useful?

33. A consumer commits every 10 seconds. What risks do you see?

Model answer: The failure window can include many processed but uncommitted records, causing replay after a crash. Depending on the application, larger batches can be acceptable or may create more duplicate work.

Cross questions: How would you choose commit strategy?

34. Auto commit vs explicit commit.

Model answer: Auto commit is simple but less explicit. Explicit commits give control over when progress is recorded and are easier to reason about for important processing semantics.

Cross questions: Does explicit commit automatically give exactly-once?

35. How would you process records concurrently?

Model answer: Use partition-level Kafka parallelism and/or controlled application worker pools. Ensure offset management respects completion ordering and desired delivery semantics.

Cross questions: What can go wrong with an uncontrolled thread pool?

36. What is backpressure in a Kafka consumer?

Model answer: Deliberately controlling intake/concurrency when downstream capacity is lower than incoming work. The goal is to prevent cascading failure even if lag increases temporarily.

Cross questions: Is increasing lag always bad?

37. A consumer receives a poison message. What happens if you retry forever?

Model answer: The same record can repeatedly fail and block progress for that partition while consuming resources. Use bounded retries and a DLT/error workflow for appropriate permanent failures.

Cross questions: How do you safely replay a DLT record?

38. How should exceptions be classified?

Model answer: Transient errors such as temporary network/DB failures can be retried; permanent validation/schema/business errors should usually not be retried indefinitely.

Cross questions: Who should own this classification?

39. What is the difference between consumer position and committed offset?

Model answer: The consumer position is where the consumer will fetch next in its current session; the committed offset is persisted group progress used for recovery/reassignment.

Cross questions: Why does this distinction matter during a crash?

40. How do you design a consumer for graceful shutdown?

Model answer: Stop accepting new work, allow in-flight work to finish within a bounded period, commit only successfully completed work according to the processing model, then leave the group cleanly.

Cross questions: What happens if shutdown takes too long?

Part III — Java / Spring Kafka Coding Problems (41–60)

41. Implement a Spring Kafka listener that acknowledges only after successful processing.

Approach / code:

```
@KafkaListener(topics = "orders", groupId = "order-service")
public void consume(OrderCreated event, Acknowledgment ack) {
    processIdempotently(event);
    ack.acknowledge();
}
```

Cross questions: Explain the acknowledgement mode, exception handling and what happens after a crash.

42. Create a resilient Kafka producer configuration.

Approach / code:

```
@Bean
ProducerFactory<String, OrderCreated> producerFactory() {
    Map<String, Object> p = new HashMap<>();
    p.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, brokers);
    p.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
    p.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, JsonSerializer.class);
    p.put(ProducerConfig.ACKS_CONFIG, "all");
    p.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);
    p.put(ProducerConfig.COMPRESSION_TYPE_CONFIG, "lz4");
    return new DefaultKafkaProducerFactory<>(p);
}
```

Cross questions: Which settings would you avoid hard-coding? What about security and delivery timeout?

43. Publish an event with a business key.

Approach / code:

```
kafkaTemplate.send(
    "order-events",
    orderId,
    event
);
```

Cross questions: Why is orderId important? What ordering guarantee do you actually get?

44. Implement idempotent DB processing.

Approach / code:

```
@Transactional
public void process(Event e) {
    if (processedEventRepository.existsById(e.id())) return;

    applyBusinessChange(e);
    processedEventRepository.save(new ProcessedEvent(e.id()));
}
```

Cross questions: Why should the uniqueness be enforced by the database too? What happens under concurrent delivery?

45. Implement retry + DLT conceptually.

Approach / code:

```
try {
    process(record);
} catch (TransientException ex) {
    // bounded retry with backoff
} catch (PermanentException ex) {
    // publish/route to DLT with original metadata
}
```

Cross questions: Why should transient and permanent exceptions be separated?

46. Write a Kafka consumer loop with explicit commit.

Approach / code:

```
while (running) {
    ConsumerRecords<String, Event> records =
        consumer.poll(Duration.ofMillis(500));

    process(records);
    consumer.commitSync();
}
```

```
}
```

Cross questions: What happens if process(records) throws before commit?

47. Implement a producer callback for failure handling.

Approach / code:

```
producer.send(record, (metadata, ex) -> {
    if (ex != null) {
        log.error("Publish failed", ex);
    } else {
        log.info("topic={}, partition={}, offset={}",
            metadata.topic(),
            metadata.partition(),
            metadata.offset());
    }
});
```

Cross questions: Would you retry inside the callback? What about application shutdown?

48. Design a Kafka event DTO.

Approach / code:

```
public record OrderCreated(
    String eventId,
    String orderId,
    Instant occurredAt,
    String schemaVersion,
    BigDecimal amount
) {}
```

Cross questions: Why include eventId and schemaVersion? What fields should not be exposed?

49. Design an outbox publisher.

Approach / code:

```
@Transactional
public void createOrder(Order order) {
    orderRepository.save(order);
    outboxRepository.save(
        OutboxEvent.forOrderCreated(order)
    );
}
// Separate publisher reads outbox and publishes to Kafka.
```

Cross questions: Why is this safer than saving to DB and publishing directly in the same service method?

50. How would you test a Kafka consumer?

Approach: Use unit tests for business logic and integration tests with an embedded/test Kafka environment or Testcontainers. Verify serialization, retries, commits, DLT and idempotency.

Cross questions: What should not be tested only with mocks?

51. How would you test duplicate delivery?

Approach: Publish the same event twice and verify the business side effect occurs once while the consumer can safely acknowledge both deliveries.

Cross questions: How would you prove the DB constraint protects against races?

52. How would you expose lag operationally from a Java service?

Approach: Prefer Kafka/platform metrics and monitoring rather than calculating lag solely inside business code. Correlate group/topic/partition lag with consumer processing and dependency metrics.

Cross questions: What alert should be based on lag slope rather than an absolute threshold?

53. How would you add correlation IDs to events?

Approach: Put a stable correlation/trace identifier in event headers or a documented envelope, propagate it through consumers and downstream calls, and avoid making it the business idempotency key unless that is its intended semantic.

Cross questions: Correlation ID vs event ID?

54. How would you handle schema evolution in Spring Kafka?

Approach: Use explicit event contracts and compatible serializers/schema tooling. Test old/new producer-consumer combinations and govern breaking changes.

Cross questions: What is backward vs forward compatibility?

55. How would you implement a Kafka Streams topology?

Approach: Define source topics, transformations, joins/aggregations and sinks; configure state stores/serialization and understand task parallelism.

Cross questions: When would you choose Streams over a normal `@KafkaListener`?

56. How would you pause consumption when a downstream system is overloaded?

Approach: Use controlled concurrency/backpressure or listener/container pause mechanisms where appropriate, while ensuring heartbeats/group stability and clear recovery logic.

Cross questions: What happens to lag while paused?

57. How would you safely batch database writes from Kafka records?

Approach: Collect records by a safe batch boundary, use a DB transaction where appropriate, process each event idempotently, and commit Kafka progress only when the batch's required work is complete.

Cross questions: What happens if record 50 fails after records 1–49 succeeded?

58. How would you preserve per-key order with asynchronous processing?

Approach: Do not allow arbitrary worker completion to reorder the same key. Partition by key and use ordered processing per partition/key or a framework that preserves the required semantics.

Cross questions: Why is a generic executor dangerous here?

59. How would you implement graceful retry without blocking a partition for hours?

Approach: Use bounded retries and an error/DLT workflow, or a retry-topic strategy with delay/backoff where supported by the chosen platform/library.

Cross questions: What is the trade-off of retry topics versus blocking retries?

Part IV — Production Incident Scenarios (61–80)

61. Lag suddenly jumps from 10K to 10M messages after a release.

Incident response: First determine whether producer rate changed or consumer throughput dropped. Compare lag by partition, processing latency, error/retry rates, consumer restarts/rebalances, CPU/GC and downstream latency. Compare the release timeline. Roll back only if evidence points to the release and rollback is safe.

Cross questions: Why not immediately add consumers?

62. Lag grows only on one partition.

Incident response: Investigate hot keys, key skew and the consumer owning that partition. Check whether one record is repeatedly failing or one downstream operation is slow. More consumers may not help because one partition cannot be actively owned by multiple group members.

Cross questions: How would you redesign the key if business ordering permits?

63. All partitions have growing lag, consumer CPU is low, DB latency is high.

Incident response: The consumer is likely waiting on the database. Inspect slow queries, locks, connection pool saturation and DB CPU. Fix DB first; scaling consumers can increase DB pressure.

Cross questions: How would you protect the DB during recovery?

64. All partitions have growing lag, consumer CPU is 95%.

Incident response: Profile CPU, inspect serialization, transformations, compression, parsing and allocation. Optimize hot paths; then scale horizontally if partitions and downstreams permit.

Cross questions: Would increasing max.poll.records help?

65. Consumer lag spikes every few minutes and then partially recovers.

Incident response: Look for periodic downstream slowness, GC pauses, batch jobs, rebalances, autoscaling churn or scheduled producer bursts. Correlate lag timestamps with application and infrastructure metrics.

Cross questions: What would you put on the incident dashboard?

66. Consumer group constantly rebalances.

Incident response: Inspect restarts, deployment churn, network instability, long processing beyond poll interval and group membership. Stabilize instances and processing first; tune timeouts only based on measurements.

Cross questions: Why can increasing max.poll.interval.ms hide rather than solve the issue?

67. Downstream payment service is down and Kafka lag is growing rapidly.

Incident response: Protect the payment service: stop uncontrolled retries, use bounded retry/backoff and circuit breaking, and accept controlled Kafka lag. Use DLT only when the message should not be retried indefinitely.

Cross questions: How do you prevent a replay storm when payment recovers?

68. One malformed event blocks a partition.

Incident response: Classify it as permanent, route it through bounded retry to DLT/error handling, alert operations, fix/correct the event if necessary and replay deliberately.

Cross questions: Why is infinite retry dangerous?

69. Broker disk reaches 90%.

Incident response: Inspect topic growth, retention, replication, partition distribution and abnormal producer volume. Confirm whether logs are growing because of legitimate retention or an incident. Increase capacity or safely adjust retention based on replay requirements.

Cross questions: What would you avoid doing?

70. Under-replicated partitions remain high for 30 minutes.

Incident response: Check broker health, disk I/O, network saturation, slow followers, broker restarts and capacity. This is a durability/availability concern, not simply a consumer issue.

Cross questions: How does ISR affect producer availability?

71. Producer latency rises while consumer processing remains normal.

Incident response: Inspect broker request latency, network, disk, leader distribution, throttling and replication. Do not scale consumers if Kafka itself is the bottleneck.

Cross questions: Which metrics distinguish broker vs application latency?

72. Consumer sees many deserialization failures after a producer deployment.

Incident response: Check schema/serialization compatibility and producer version. Route invalid records appropriately and restore compatible contract. Do not silently discard data.

Cross questions: How would schema governance prevent recurrence?

73. A new consumer version is deployed and lag doubles.

Incident response: Compare processing latency, downstream calls, CPU/GC, serialization and error/retry rates against the old version. Use canary/controlled rollout where possible.

Cross questions: Would you immediately roll back? What evidence would you want?

74. Consumer lag is high but business latency is acceptable.

Incident response: Understand whether lag consists of low-priority/replay traffic, whether consumers are intentionally throttled, and whether the actual business SLA is being met. Do not optimize a metric without its business meaning.

Cross questions: What is a better alert than a raw lag threshold?

75. Lag is low but customers report delayed processing.

Incident response: Lag may not represent end-to-end business latency. Inspect timestamps in events, consumer processing time, downstream queueing and external service latency. Build an end-to-end latency metric.

Cross questions: Why can offset lag be misleading?

76. A consumer instance has huge memory usage after increasing max.poll.records.

Incident response: Large batches can increase object retention and processing memory. Reduce batch size, profile allocation and consider bounded concurrency.

Cross questions: How do you choose batch size empirically?

77. Increasing consumer instances makes the DB fail.

Incident response: Consumer concurrency exceeded DB capacity. Reduce concurrency, tune the DB/pool, batch operations or use backpressure. Kafka scaling must respect downstream capacity.

Cross questions: What is the formula you use mentally for safe concurrency?

78. Partition count is 6, consumer instances are 30, and lag remains high.

Incident response: Extra consumers cannot increase partition-level parallelism. Check processing rate and consider increasing partitions only after reviewing key/order design and downstream capacity.

Cross questions: What happens to existing records when partitions are increased?

79. A producer sends the same business event twice with different event IDs.

Incident response: Kafka idempotence cannot detect semantically duplicated application events if the producer created two distinct records. Use a stable business idempotency key or deterministic event identity at the business layer.

Cross questions: What is the difference between transport duplicate and business duplicate?

Part V — System Design & Architecture Scenarios (81–90)

81. Design an order-processing platform using Kafka.

Design answer: Use Order Service + PostgreSQL, transactional outbox, Kafka order-events, and independent consumer groups for payment, notification and analytics. Partition by orderId if per-order ordering is required. Consumers are idempotent. Add schema governance, DLT, observability and security.

Cross questions: Why outbox? Where does exactly-once apply?

82. Design DB → Kafka integration with no lost events.

Design answer: Write business state and outbox event in one DB transaction. A separate publisher reads unpublished outbox rows and publishes to Kafka. Mark publication state safely. Consumers remain idempotent.

Cross questions: What if publisher crashes after Kafka publish but before marking the row sent?

83. Design a payment event pipeline with strict ordering per account.

Design answer: Use accountId as key and enough partitions for expected concurrency. Keep all account events in the same partition. Ensure consumers do not reorder asynchronous work for the same partition/key.

Cross questions: What if one account is extremely hot?

84. Design for 1 million events/sec.

Design answer: Start with workload assumptions: average/peak bytes, partitions, broker capacity, replication, network, storage and consumer processing. Partition count must support parallelism, but capacity must be benchmarked. Use batching/compression and multiple brokers/failure domains.

Cross questions: How would you capacity-test it?

85. Design replay for a failed consumer version.

Design answer: Keep adequate topic retention. Deploy a corrected consumer group or reset/replay offsets carefully. Isolate replay from live processing when necessary to avoid overwhelming downstreams.

Cross questions: How do you prevent replay from creating duplicate business effects?

86. Design multi-team event contracts.

Design answer: Use domain-owned schemas, compatibility rules, version governance, documentation, ownership and automated compatibility checks. Avoid leaking internal DB models.

Cross questions: Who owns breaking changes?

87. Design Kafka across availability zones.

Design answer: Distribute brokers and replicas across failure domains, use appropriate replication factor and placement, and understand network/latency implications. Test broker/AZ failure scenarios.

Cross questions: What changes in capacity planning?

88. Design a notification service that can tolerate email provider outages.

Design answer: Consume events at-least-once, make notification requests idempotent, use timeout/circuit breaker, bounded retry, backpressure and DLT where appropriate. Preserve enough retention for recovery/replay.

Cross questions: Why not retry forever?

89. Design Kafka for audit/event history.

Design answer: Use durable event contracts, appropriate retention/storage, replication, access controls, schema governance and immutable-event principles. Consider PII/security and legal retention requirements.

Cross questions: How does compaction affect audit use cases?

90. Design a migration from IBM MQ/JMS to Kafka.

Design answer: First identify message semantics, ordering, transactions, selectors/routing, retry/DLQ behavior, consumer patterns and operational ownership. Introduce Kafka alongside MQ, dual-publish or bridge carefully, validate equivalence, migrate consumers incrementally, then decommission MQ.

Cross questions: How do you avoid duplicate processing during migration?

Part VI — Leadership, Architecture Review & Trick Questions (91–100)

91. A team proposes Kafka for every synchronous API call. How do you respond?

Lead answer: Challenge the requirement. Kafka is excellent for asynchronous event streaming but does not automatically improve request/response latency or transactional semantics. Use synchronous APIs where immediate response is required and Kafka where decoupling/event-driven behavior provides value.

Cross questions: What questions would you ask before rejecting the design?

92. A developer says “acks=all means zero data loss.” Correct them.

Lead answer: acks=all is a strong producer acknowledgement setting, but zero-loss guarantees depend on replication, min.insync.replicas, failure modes, producer behavior and operational configuration. It is not a blanket guarantee.

Cross questions: What role does RF play?

93. A developer says “Kafka is exactly once.” Correct them.

Lead answer: Kafka supports transactions and exactly-once semantics for compatible Kafka workflows, but arbitrary external side effects are not automatically exactly-once. DB/API interactions need idempotency or an appropriate integration pattern.

Cross questions: Give a DB example.

94. A team wants 5,000 partitions immediately for future growth.

Lead answer: Ask for actual throughput, consumer parallelism, retention, broker capacity, recovery expectations and growth assumptions. Partition count should be capacity-driven rather than speculative because partitions have operational cost.

Cross questions: What future-scaling strategy could be used instead?

95. A team uses infinite retries because “we cannot lose messages.”

Lead answer: Infinite retries can create a stuck partition and cascading outage. Durability should be achieved through retention/replay, bounded retries, DLT/error workflows and idempotent processing—not by blocking forever.

Cross questions: How would you explain this to business stakeholders?

96. A consumer is slow because an external API takes 10 seconds. The team proposes 100 consumers.

Lead answer: First establish downstream rate limits and capacity. 100 consumers can multiply pressure and cause an outage. Use timeouts, circuit breaker, backpressure, controlled concurrency and potentially batching/caching.

Cross questions: How do you determine safe concurrency?

97. What is your approach to a Kafka design review?

Lead answer: Check business semantics, partition key/order, throughput, failure handling, delivery semantics, replication, schema evolution, security, observability, capacity, deployment strategy and recovery testing.

Cross questions: Which areas are most often missed?

98. How do you communicate a Kafka incident to leadership?

Lead answer: State impact, current customer/business effect, suspected root cause, containment, recovery estimate, risks and next update. Avoid jargon without context. Separate facts from hypotheses.

Cross questions: Give an example of a concise update.

99. When should a Tech Lead reject a Kafka-based solution?

Lead answer: When Kafka does not match the requirement—for example strict synchronous response, simple low-volume CRUD, unnecessary operational complexity, or when downstream semantics cannot tolerate asynchronous eventual consistency.

Cross questions: What alternative would you suggest?

100. What differentiates a Senior Developer from a Kafka Tech Lead?

Lead answer: A senior developer can implement producers/consumers. A Tech Lead can choose semantics, design partitioning and failure handling, size the system, protect dependencies, troubleshoot incidents, govern contracts and explain trade-offs to technical and business stakeholders.

Cross questions: Give a concrete example from a production incident.

Production Incident Playbook

Use this sequence when a Kafka-based Java service is reported as “slow” or “backlogged.”

- **1. Scope:** topic, consumer group, time window, business impact.
- **2. Trend:** current lag, lag slope, recovery slope and time-to-recover.
- **3. Partition shape:** all partitions vs hot partitions.
- **4. Rate comparison:** producer records/sec vs consumer records/sec.
- **5. Consumer health:** CPU, GC, memory, processing latency, exceptions, retries, restarts.
- **6. Group stability:** rebalances, poll interval, membership churn.
- **7. Downstream:** DB/API latency, pool utilization, locks, rate limits.
- **8. Kafka health:** broker CPU/disk/network, fetch latency, under-replicated/offline partitions.
- **9. Containment:** protect saturated dependencies; stop retry storms.
- **10. Fix:** apply the smallest safe change.
- **11. Verify:** lag slope must reverse while dependencies remain healthy.
- **12. Prevent:** root-cause analysis, alerting, capacity planning and runbook updates.

Kafka Troubleshooting Matrix

Observation	Likely area	First investigation	Common wrong fix
All partitions lagging	Consumer/downstream/traffic	Rates + processing latency	Blind consumer scaling
One partition lagging	Key skew/poison record	Partition lag + key/failure	Add consumers
CPU high	Application	Profile/GC/serialization	Increase poll batch
CPU low + DB slow	Database	Query/pool/locks	Scale consumers
API slow	Downstream	Latency/timeouts/retries	Infinite retry
Frequent rebalances	Group stability	Restarts/poll interval	Only increase timeout
Fetch latency high	Kafka/network	Broker/network/disk	Scale application
Under-replicated	Kafka durability	Broker/network/disk	Tune consumer
Disk high	Storage/retention	Topic growth/retention/RF	Delete blindly

Interview Rapid-Fire Revision

- **Ordering?** Within a partition.
- **Parallelism?** Partitions are the main unit.
- **Consumer group?** Share partition work and offsets.
- **Duplicate after crash?** At-least-once behavior; make business processing idempotent.
- **acks=all?** Strong producer acknowledgement; not exactly-once.
- **Idempotent producer?** Protects against duplicate producer retries, not arbitrary business duplicates.
- **Lag?** Symptom; investigate trend, partition shape and bottleneck.
- **Hot partition?** Key/traffic skew; more consumers alone may not help.
- **Slow DB?** Fix/protect DB before scaling Kafka consumers.
- **Slow API?** Timeout, circuit breaker, bounded retry and backpressure.
- **Poison record?** Bounded retry + DLT/error workflow.
- **DB + Kafka consistency?** Transactional outbox + idempotent consumer is a common robust pattern.
- **Exactly-once?** Scoped; external side effects still need design.

- **Tech Lead answer?** Requirement → design → failure → trade-off → observability → recovery.

30-Day Kafka Interview Practice Plan

Days	Focus	Practice
1–5	Core model	Explain topics, partitions, offsets, groups aloud
6–10	Producer/consumer	Write Java producer/consumer; explain commits/retries
11–15	Reliability	Replication, ISR, acks, idempotence, delivery semantics
16–20	Spring Kafka	Listener, error handling, DLT, testing, schema
21–24	Troubleshooting	Solve lag, rebalances, DB/API and broker incidents
25–27	System design	Outbox, payment/order, replay, capacity
28–29	Mock interview	Answer 20 scenarios aloud with cross questions
30	Final review	Give a 10-minute Kafka architecture walkthrough

Final Tech Lead Checklist

- I can explain Kafka internals without memorized definitions.
- I can choose a partition key from business ordering requirements.
- I can explain what happens when producer/consumer/broker/downstream fails.
- I can distinguish transport duplicates from business duplicates.
- I can design at-least-once + idempotent processing.
- I can explain Kafka transactions and their limits.
- I can design retry/DLT/backpressure without creating a retry storm.
- I can diagnose consumer lag systematically.
- I can reason about partition count and safe consumer concurrency.
- I can design DB + Kafka using an outbox pattern.
- I can discuss schema evolution and event ownership.
- I can lead a Kafka production incident and communicate impact clearly.
- I can defend trade-offs instead of giving configuration-only answers.

Final Memory Framework

Requirement → Key → Partition → Group → Offset → Delivery Semantics → Failure Handling → Replication → Schema → Capacity → Observability → Recovery → Trade-off.

If you can walk an interviewer through that chain with a concrete Java/Spring example, you are answering at Tech Lead level rather than simply demonstrating Kafka API knowledge.